Each order request is submitted for a specific ISBN. Order Service needs to know the book's ISBN, title, author, and price to process an order correctly. Currently, Catalog Service exposes a `/books/{bookIsbn}` endpoint that returns all the available information about a book. In a real scenario, you might expose a different endpoint that returns an object containing only the required information (a DTO). For the sake of this example, we'll reuse the existing endpoint, since our focus right now is on building reactive clients.

Having established which endpoint to call, how should you model the exchange between the two applications? You've just reached a crossroad:

- *Create a shared library*—One option is to create a shared library with the classes used by both applications, and import it as a dependency into both projects. As per the 15-factor methodology, such a library would be tracked in its own codebase. Doing so would ensure that the model used by both applications is consistent and never out of sync. However, it would mean adding implementation coupling.
- *Duplicate the class*—The other option is to replicate the class into the upstream application. By doing so, you wouldn't have implementation coupling, but you would have to take care of evolving the replicated model as the original one changes in the downstream application. There are a few techniques like consumer-driven contracts that can identify, through automated tests, when the called API changes. Besides checking the data model, those tests would also verify other aspects of the exposed API, like HTTP methods, response statuses, headers, variables, and so on. I will not cover this subject here, but I recommend checking the Spring Cloud Contract project if you're interested (https:// spring.io/projects/spring-cloud-contract).